

SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

Bachelor of Technology
in
Computer Science & Engineering
By

KAMISETTY LEELA LASYA PRIYA(VTU26551)

10211CS224
Full Stack Application Development
Slot : E2-02



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “**Smart Campus Event Management System**” by KAMISSETTY LEELA LASYA PRIYA (VTU26551) has been carried out in the Winter Semester of Academic Year 2025–2026 (WS2526).

Signature of Faculty

Dr.S.HEMAMALINI

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra. M

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

KAMISETTY LEELA LASYA PRIYA

Date:06/05/2026

ABSTRACT

The Smart Campus Event Management System is a web-based application designed to automate and simplify the management of campus events, workshops, and seminars in educational institutions. Traditional event management methods are often manual, time-consuming, and prone to errors in handling registrations, updates, and communication.

This system provides a centralized platform where students can securely register, log in, browse upcoming events, register for events, and provide feedback. Administrators can create, update, delete, and monitor events through a dedicated dashboard.

The application includes features such as event scheduling, registration deadline management, department-based filtering, announcements, and email notifications. Students receive event details and registration confirmations directly to their registered email IDs.

Overall, the Smart Campus Event Management System offers a scalable, secure, and efficient digital solution for managing campus events while reducing manual effort and improving productivity.

Keywords: Smart Campus, Event Management, Spring Boot, Thymeleaf, MySQL, Automation.

LIST OF FIGURES

1.1	Framework Architecture	3
3.1	Front End Architecture	9
3.2	Backend Architecture	10
4.1	Project Architecture	12
4.2	Data Flow Diagram	13
5.1	Login Page - Admin/Student Authentication	16
5.2	Student Dashboard - View and Register Events	17
5.3	Admin Dashboard - Event Management and Registra- tions	18

LIST OF TABLES

3.1	Database Tables Used in Smart Campus Event Management System	11
7.1	Mapping of Project to Complex Engineering Problem (CEP)	21
7.2	Mapping of Project to Complex Engineering Activities (CEA)	22
7.3	SDG Mapping for the Project	23

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
MVC	Model View Controller
ORM	Object Relational Mapping
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6

3.2	Backend Implementation	7
3.3	Database Implementation	10
4	METHODOLOGIES	12
4.1	Project Architecture	12
4.2	Data Flow Diagram	13
4.3	Module 1: User Management Module	14
4.4	Module 2: Event Management Module	14
4.5	Module 3: Notification and Feedback Module	14
5	RESULTS AND DISCUSSIONS	16
6	CONCLUSION AND FUTURE ENHANCEMENTS	19
6.1	Conclusion	19
6.2	Future Enhancements	19
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	20
7.1	Mapping of the Project to Complex Engineering Prob- lem (CEP)	21
7.2	Mapping of the Project to Complex Engineering Ac- tivities (CEA)	22
7.3	SDG Mapping (CEP Section)	23

8	SOURCE CODE	24
8.1	Source Code	24
9	REFERENCES	33

Chapter 1

INTRODUCTION

1.1 Introduction

The Smart Campus Event Management System is a web-based application developed to simplify and automate the management of campus events, workshops, and seminars in educational institutions. Traditional event management processes are often handled manually, which can lead to inefficiencies, communication delays, and errors in registration handling.

This system provides a centralized platform where students can securely register, log in, browse upcoming events, register for events, and provide feedback. Administrators can efficiently manage the complete event lifecycle by creating, updating, deleting, and monitoring events through a dedicated admin dashboard.

The system also includes advanced features such as event registration tracking, announcements, email notifications, department-based filter-

ing, analytics, and registration deadline management. By integrating these functionalities into a single platform, the application improves efficiency, reduces manual effort, and ensures better communication between students and administrators.

Overall, the Smart Campus Event Management System offers a scalable, secure, and professional solution for managing campus events in a digital environment.

1.2 Aim of the Project

The aim of this project is to develop a web-based Smart Campus Event Management System to automate the management of campus events, workshops, and seminars. It provides a centralized platform for students to register and participate in events, while admins can efficiently create, update, and manage events. The system reduces manual work, improves communication, and ensures better event handling.

1.3 Scope of the Project

The scope of this project includes managing campus events through a centralized web platform. Students can register, view, and participate in events, while admins can manage event creation, updates, and registrations. The system can be further enhanced with mobile support, cloud deployment, and advanced analytics.

1.4 Framework

The Smart Campus Event Management System is developed using Spring Boot with a layered architecture following the Model-View-Controller (MVC) design pattern. The frontend is designed using HTML5, CSS3, and Thymeleaf, while the backend uses Spring MVC and Spring Data JPA for business logic and database operations. MySQL is used for storing event, user, and registration details, ensuring efficient data management.

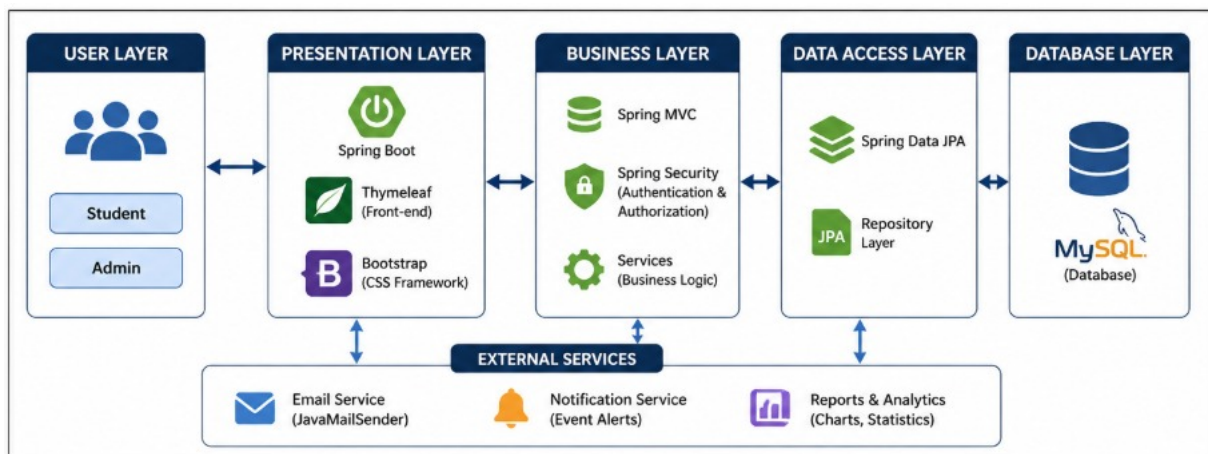


Figure 1.1: Framework Architecture

Figure 1.1 illustrates the framework architecture of the Smart Campus Event Management System. It shows the interaction between frontend, backend, and database layers. The architecture ensures modularity, scalability, and smooth system communication.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The Smart Campus Event Management System is developed after analyzing several existing event management applications available in the market. These applications provide event organization and registration features but lack certain campus-specific functionalities required by educational institutions.

1. Eventbrite: Eventbrite is a widely used online event management platform that allows users to create, promote, and manage events. It supports ticket booking, attendee management, and email notifications. However, it is mainly designed for commercial events and does not provide institution-specific features such as student login, department-based filtering, or academic event management. Website: <https://www.eventbrite.com>

2. Whova: Whova is an event management and networking platform that offers event registration, agenda scheduling, attendee networking, live announcements, and feedback collection. It is suitable for con-

ferences and seminars but is more focused on large-scale professional events rather than campus event workflows. Website: <https://whova.com>

3. Cvent: Cvent is an enterprise-level event management software that supports event planning, venue selection, registration management, analytics, and email marketing. Although feature-rich, it is costly and complex for small or medium educational institutions. Website: <https://www.cvent.com>

4. RSVPify: RSVPify provides online invitation, RSVP tracking, ticketing, and guest management features. It is simple and user-friendly but lacks academic modules such as student authentication, department categorization, and campus announcement integration. Website: <https://rsvpify.com>

Comparison and Need for Proposed System

Existing systems mainly target business events, conferences, and commercial event management. They do not fully address campus-specific needs such as student registration using institutional credentials, department-wise event categorization, academic notifications, and simplified administrator control. Hence, the Smart Campus Event Management System is proposed as a customized solution specifically designed for educational institutions.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Smart Campus Event Management System is developed using HTML5, CSS3, and Thymeleaf. HTML5 is used to structure web pages, CSS3 is used for styling and responsive layouts, and Thymeleaf is used as the server-side template engine to dynamically render data from the backend. The frontend is executed through a web browser and integrated with Spring Boot.

3.1.2 Structure of the Project

- The frontend includes multiple pages such as:
 1. Login page
 2. Registration page
 3. Student dashboard
 4. Admin dashboard

5. Event listing page

6. Event details page

7. Feedback forms

- All HTML templates are organized within the `templates` folder
- Static resources, including CSS files, are stored in the `static` folder

3.1.3 Execution Procedure

The frontend is loaded through Spring Boot application startup. Users access the application through browser using localhost. Requests from users are routed to controllers, which process data and return Thymeleaf templates for rendering.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Java with Spring Boot framework. Spring MVC handles request mapping and controller logic, Spring Data JPA manages database operations, and Spring Security provides authentication and authorization. MySQL is used as the database, and Maven is used for dependency management and build automation.

3.2.2 Structure of the Project

- The backend follows a layered architecture
- It is organized into the following packages:
 1. Controller
 2. Service
 3. Repository
 4. Model
 5. DTO (Data Transfer Object)
 6. Config
 7. Exception
- Controllers handle HTTP requests and responses
- Services manage the business logic of the application
- Repositories interact with the database
- Models represent database entities
- This structured architecture helps to:
 1. Improve maintainability
 2. Enhance scalability
 3. Promote code reusability
- Overall, it supports a cleaner and more efficient application development process

3.2.3 Execution Procedure

The backend execution begins when the Spring Boot application is started using Maven commands. The embedded Tomcat server initializes the application and loads all required dependencies. User requests are received through controllers, processed by service classes, and database operations are performed using repositories connected to MySQL. Finally, the response is rendered to the frontend using Thymeleaf templates.

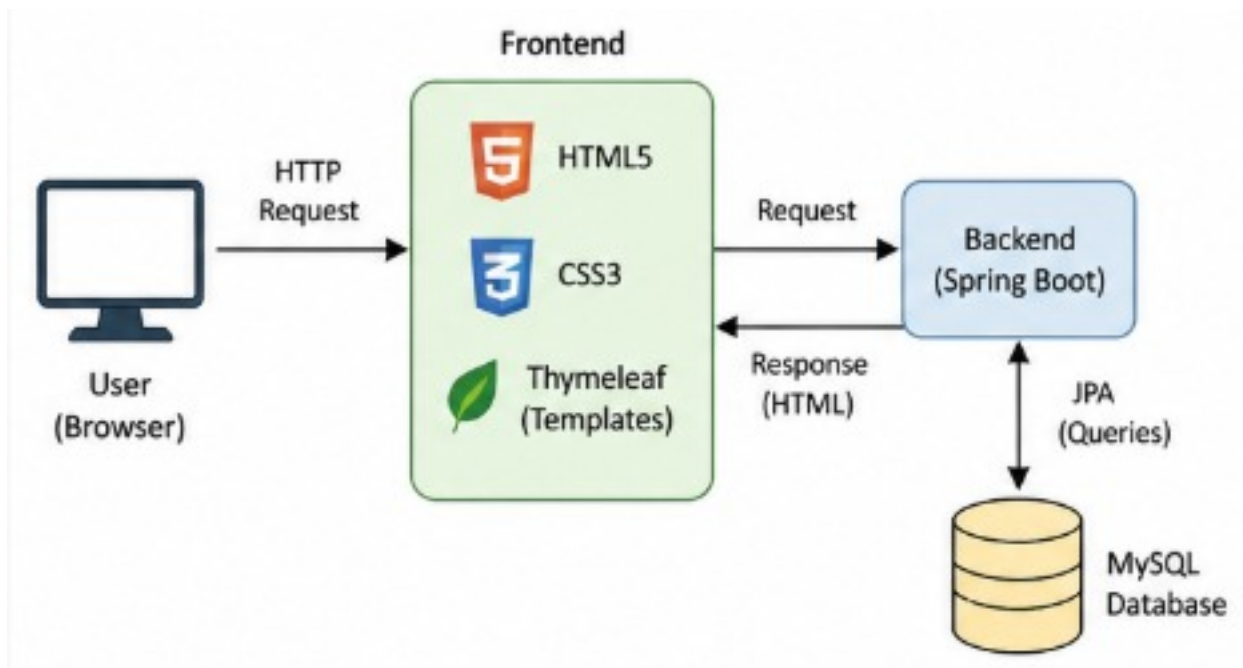


Figure 3.1: Front End Architecture

Figure 3.1 shows the frontend architecture and user interface flow of the system. It ensures smooth navigation and interaction for users.

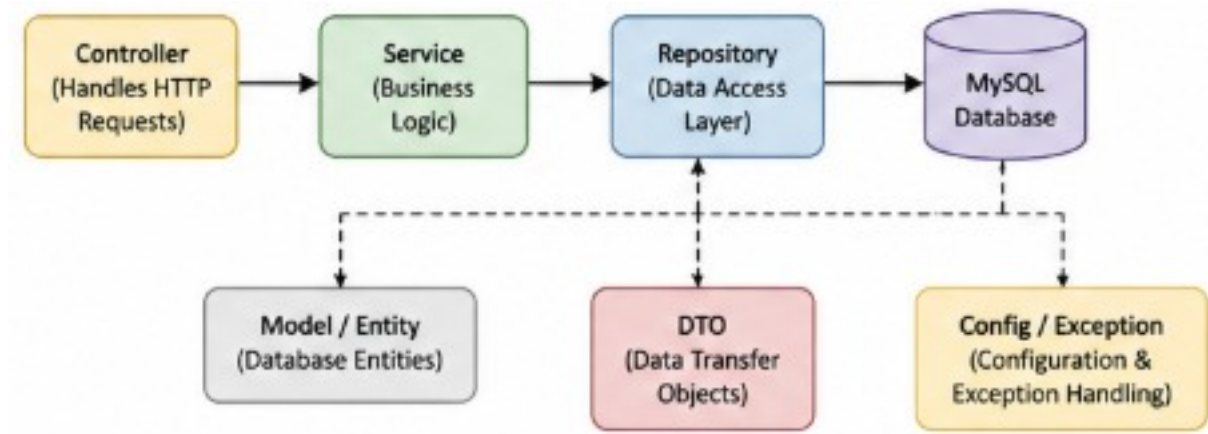


Figure 3.2: Backend Architecture

Figure 3.2 illustrates the backend modules and database connectivity. It manages request processing and business logic execution.

3.3 Database Implementation

3.3.1 Database Configuration

The Smart Campus Event Management System uses MySQL as the relational database management system. Database connectivity is established using Spring Data JPA and JDBC configuration in the application.properties file. The database stores all application data securely, including users, events, registrations, and feedback information.

3.3.2 Schema Structure

The database schema is designed using entity relationships between multiple tables. Core entities include User, Event, Registration, and Feedback. Relationships are maintained using primary keys and for-

eign keys to ensure data consistency and integrity.

3.3.3 Tables created

The main tables created in the database are users, admins, events, registrations, and feedback. These tables store login credentials, event details, registration records, participant information, and student feedback data required for the smooth functioning of the system.

S.No	Table Name	Purpose
1	Users	Stores student/user login details and profile information
2	Admins	Stores administrator credentials and access details
3	Events	Contains event details such as title, date, venue, and description
4	Registrations	Maintains records of student event registrations
5	Feedback	Stores participant feedback and suggestions after events

Table 3.1: Database Tables Used in Smart Campus Event Management System

Table 3.1 presents the major database tables used in the system.

These tables are responsible for storing user, event, registration, and feedback details.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Smart Campus Event Management System follows a layered MVC architecture consisting of presentation layer, controller layer, service layer, repository layer, and database layer. This architecture ensures modularity, maintainability, and efficient communication between components.

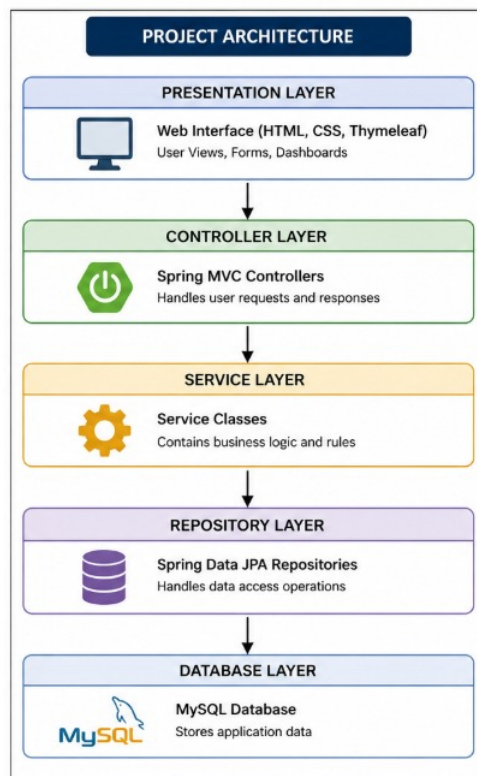


Figure 4.1: Project Architecture

Figure 4.1 illustrates the overall architecture of the Smart Campus Event Management System. It shows the interaction between presentation, business logic, and database layers.

4.2 Data Flow Diagram

The data flow diagram illustrates how user requests are processed through the system. Students and admins interact with the application, requests are processed by the backend, and data is stored or retrieved from the database.

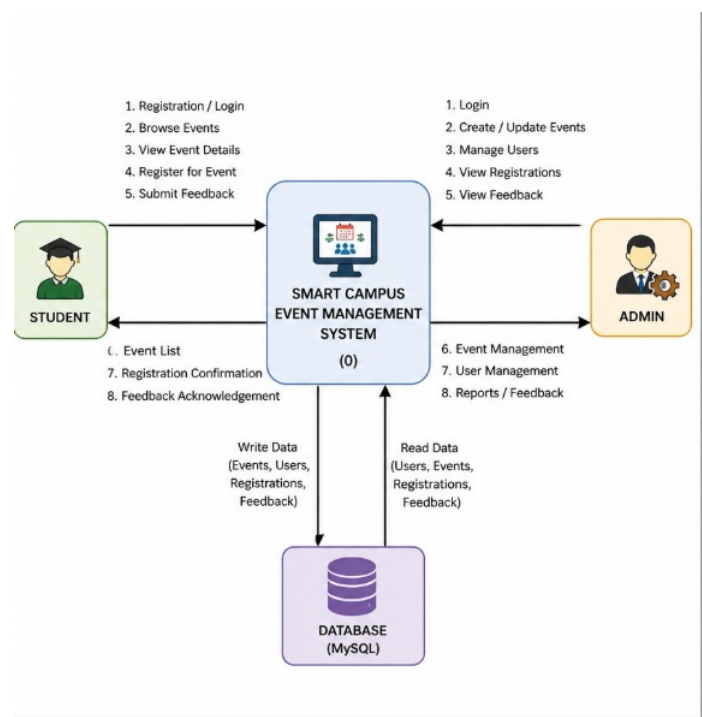


Figure 4.2: Data Flow Diagram

Figure 4.2 represents the flow of data between users, application modules, and database. It explains how requests are processed and responses are generated. This diagram ensures a clear understanding of system workflow.

4.3 Module 1: User Management Module

This module is responsible for managing user registration and authentication within the system. It allows different types of users such as students, faculty, and administrators to create accounts. Secure login functionality is implemented using user credentials like username and password. The module ensures data protection through proper validation and authentication mechanisms. It supports role-based access control to restrict or allow access to specific features. Users are assigned roles that define their permissions within the system. The module provides options for users to update their profile information. It also includes a password reset feature to recover forgotten credentials. Personal information management is made easy and user-friendly. Overall, this module enhances system security and ensures controlled access for all users.

4.4 Module 2: Event Management Module

This module allows administrators and event organizers to create, update, and delete campus events. It includes event details such as title, description, date, time, venue, and capacity. Students can view available events and register for participation. The system also prevents duplicate registrations and manages event schedules efficiently.

The module maintains proper event records and participant information. It ensures smooth coordination between organizers and students. Event schedules are managed systematically to avoid conflicts. This module improves overall event planning and execution.

4.5 Module 3: Notification and Feedback Module

This module is responsible for sending real-time notifications and alerts regarding events, reminders, and updates through email or sys-

tem notifications. After event completion, users can provide feedback and ratings. This helps in improving the quality of future events and maintaining engagement between organizers and participants.

The notification system keeps users informed about important announcements. Feedback collection helps organizers analyze participant satisfaction. Ratings and suggestions are stored for future improvements. This module enhances communication and user engagement within the system.

Advantages of this Project

- **Efficient Event Management:** Automates event creation, registration, and scheduling, reducing manual effort.
- **Improved Communication:** Provides instant notifications and alerts to students and faculty.
- **Centralized System:** Maintains all event data in a single platform for easy access and management.
- **Reduced Manual Work:** Minimizes human errors and repetitive administrative tasks.
- **Better Participation:** Encourages students to participate more in campus events.

Disadvantages

- Requires stable internet connection for proper functioning.
- Initial setup and maintenance require technical expertise.
- System performance may reduce under high traffic.
- Users may need initial training to understand system features.

Chapter 5

RESULTS AND DISCUSSIONS

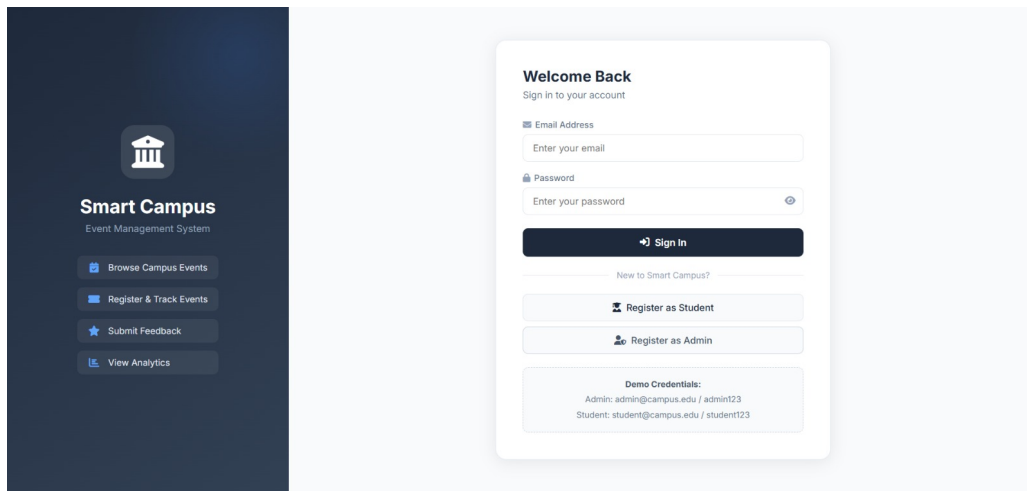


Figure 5.1: Login Page - Admin/Student Authentication

This figure 5.1 shows the login page where both students and administrators enter their credentials to access the system. Based on the role, users are redirected to their respective dashboards.

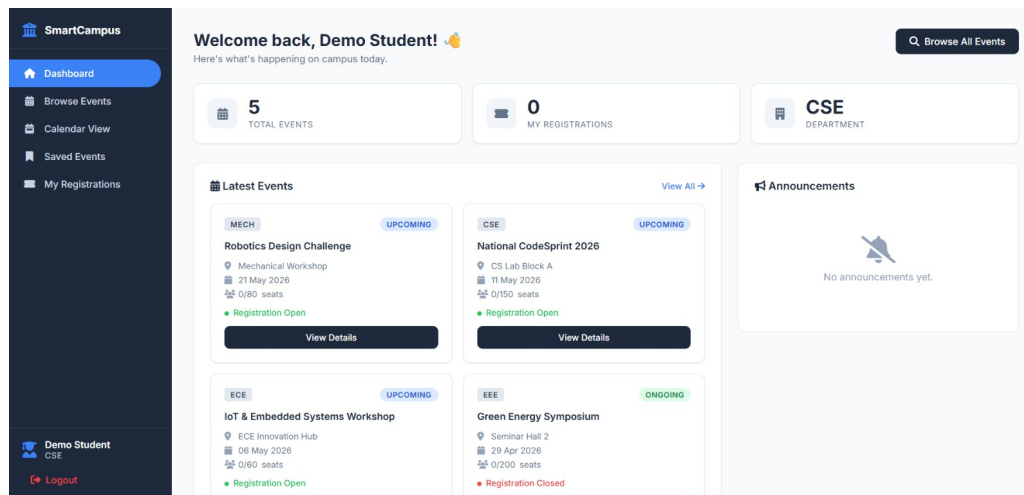


Figure 5.2: Student Dashboard - View and Register Events

This figure 5.2 represents the student dashboard where students can view all available campus events, check event details, and register for participation.

- Students can search and browse available events easily.
- Students can register for events using their email ID.
- Students receive a confirmation email after successful registration.

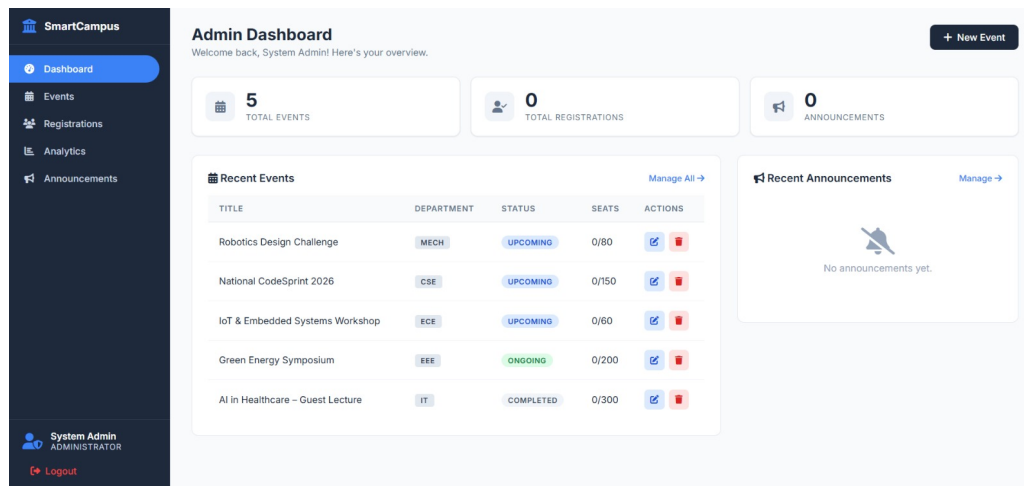


Figure 5.3: Admin Dashboard - Event Management and Registrations

This figure 5.3 shows the admin dashboard where the administrator can create new events, update or delete events, and view student registration details.

- Admin can create and manage all campus events including date, time, venue, and description.
- Admin can view and monitor student registrations for each event.
- Admin can track overall event participation and manage registrations efficiently.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System has been successfully designed and developed to simplify the management of campus events. The system provides an efficient platform for handling user registration, event creation, event participation, notifications, and feedback collection. It reduces manual effort, improves communication between students and administrators, and ensures better organization of events. By centralizing all event-related activities, the system enhances accessibility, accuracy, and overall efficiency in managing campus events.

6.2 Future Enhancements

In the future, the system can be improved by adding mobile application support for better accessibility. Integration with SMS and WhatsApp notifications can also be implemented for faster communication. Artificial Intelligence-based event recommendations can be added to suggest events based on user interests. Additionally, online payment gateways can be integrated for paid events, and advanced analytics dashboards can be included to analyze student participation .

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of knowledge	Requires in depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge in Spring Boot, MVC architecture, database design, REST APIs, and frontend technologies for scalable event management.
WP2	Conflicting requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and security with secure authentication and data validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Includes system design, schema creation, validation handling, and registration mechanisms.
WP4	Familiarity	Partially new problems	WK4, WK8	Real-time updates and analytics introduce challenges beyond manual systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Custom implementation for filtering, tracking, and statistics generation.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Addresses students, faculty coordinators, and administrators.
WP7	Interdependence	System level integration	WK3, WK5, WK6	Integrates UI, backend, database, validation, and security modules.

21
Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases.
EA2	Level of Interactions	Complex interactions between modules	Handles event management, registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Web-based centralized platform with filtering and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Improves engagement and institutional efficiency while reducing errors.
EA5	Familiarity	Beyond standard practices	Combines development, security, and analytics in one system.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates participation in academic events, workshops, and seminars.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives and resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling.
SDG 12	Responsible Consumption and Production	Reduces paper usage through online registration and management.

Table 7.3: SDG Mapping for the Project

Chapter 8

SOURCE CODE

8.1 Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login - Smart Campus</title>
  <link rel="stylesheet" href="styles.css">
  <style>
    .auth-container {
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 80vh;
    }
    .auth-card {
      width: 100%;
      max-width: 450px;
    }
    .toggle-text {
      text-align: center;
      margin-top: 20px;
    }
  </style>
</head>
<body>
  <div class="auth-container">
    <div class="auth-card">
      <div class="login-form">
        <div class="login-form__header">
          <h1>Smart Campus</h1>
          <h2>Login</h2>
        </div>
        <div class="login-form__body">
          <div class="login-form__input">
            <input type="text" value="Username" />
          </div>
          <div class="login-form__input">
            <input type="password" value="Password" />
          </div>
          <div class="login-form__button">
            <button>Login</button>
          </div>
          <div class="login-form__text">
            <p>Don't have an account? <a href="#">Sign Up</a></p>
          </div>
        </div>
      </div>
      <div class="toggle-text">
        <p>Remember me</p>
      </div>
    </div>
  </div>
</body>
</html>
```

```

    }
</style>
</head>
<body>

<div class="container auth-container">

  <!-- Login Form -->
  <div class="auth-card" id="login-box">
    <h2>Welcome Back</h2>
    <form>
      <label>Email</label>
      <input type="email" placeholder="your@email.com" required>

      <label>Password</label>
      <input type="password" placeholder="*****" required>

      <button type="submit">Sign In</button>
    </form>
  </div>

  <!-- Register Form -->
  <div class="auth-card" id="register-box" style="display:none;">
    <h2>Create Account</h2>
    <form>
      <label>Full Name</label>
      <input type="text" placeholder="John Doe" required>

      <label>Email</label>
      <input type="email" placeholder="your@email.com" required>

      <label>Password</label>
      <input type="password" placeholder="*****" required>

      <button type="submit">Register</button>
    </form>
  </div>
</div>

```

```

        </form>
    </div>

</div>

</body>
</html>

package com.smartcampus.events;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class SmartEventsApplication {

    public static void main(String[] args) {
        SpringApplication.run(SmartEventsApplication.class, args);
    }
}

POM.XML
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.4</version>
        <relativePath/>
    </parent>

```

```
<groupId>com.smartcampus</groupId>
<artifactId>smarteventssystem</artifactId>
<version>1.0.0</version>
<packaging>jar</packaging>
<name>Smart Campus Event Management System</name>
<description>A full-stack Spring Boot application for managing campus
    events</description>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>
    <!-- Spring Web MVC -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Data JPA -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- Spring Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- Thymeleaf -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<!-- Thymeleaf Spring Security Extras -->
<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>

<!-- Bean Validation -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- H2 In-Memory Database -->
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- Lombok -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>

<!-- DevTools -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>

```

```

</dependency>

<!-- Spring Boot Mail for Notifications -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>

<!-- PDF generation for participation certificates -->
<dependency>
    <groupId>com.github.librepdf</groupId>
    <artifactId>openpdf</artifactId>
    <version>1.3.39</version>
</dependency>

<!-- Spring Boot Test -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>

<!-- Spring Security Test -->
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
</dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>

```

```

        <configuration>
            <excludes>
                <exclude>
                    <groupId>org.projectlombok</groupId>
                    <artifactId>lombok</artifactId>
                </exclude>
            </excludes>
        </configuration>
    </plugin>
</plugins>
</build>
</project>
announcement.java
package com.smartcampus.events.service;

import com.smartcampus.events.model.Announcement;
import com.smartcampus.events.model.Event;
import com.smartcampus.events.model.User;
import com.smartcampus.events.repository.AnnouncementRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@RequiredArgsConstructor
public class AnnouncementService {

    private final AnnouncementRepository announcementRepository;

    @Transactional
    public Announcement createAnnouncement(String title, String message,
        User admin, Event event) {
        Announcement ann = Announcement.builder()

```



```

        .title(title)
        .message(message)
        .postedBy(admin)
        .event(event)
        .build();
    return announcementRepository.save(ann);
}

public List<Announcement> findAll() {
    return announcementRepository.findAllByOrderByCreatedAtDesc();
}

public List<Announcement> findRecent() {
    return announcementRepository.findTop5ByOrderByCreatedAtDesc();
}
}

custom user details
package com.smartcampus.events.service;

import com.smartcampus.events.model.User;
import com.smartcampus.events.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.authority.SimpleGrantedAuthority
    ;
import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

```

```
@Override
public UserDetails loadUserByUsername(String email) throws
    UsernameNotFoundException {
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new UsernameNotFoundException("No user
            found with email: " + email));

    return org.springframework.security.core.userdetails.User.builder()
        .username(user.getEmail())
        .password(user.getPassword())
        .authorities(List.of(new SimpleGrantedAuthority("ROLE_" +
            user.getRole().name())))
        .build();
}
}
```

Chapter 9

REFERENCES

1. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill, 2020.
2. I. Sommerville, *Software Engineering*, 10th ed., Pearson Education, 2016.
3. C. S. Horstmann, *Core Java Volume I—Fundamentals*, 11th ed., Pearson, 2018.
4. K. Siva Prasad Reddy, *Spring Boot Microservices and Web Applications*, BPB Publications, 2022.
5. G. Deitel and P. Deitel, *Java: How to Program*, 11th ed., Pearson, 2018.
6. J. Bloch, *Effective Java*, 3rd ed., Addison-Wesley, 2018.
7. Spring Framework Documentation, “Spring Boot Reference Guide.” Available: <https://spring.io/projects/spring-boot>
8. Oracle Corporation, “Java Documentation.” Available: <https://docs.oracle.com/javase/>
9. MySQL Documentation, “MySQL Reference Manual.” Available: <https://dev.mysql.com/doc/>
10. Bootstrap Documentation, “Bootstrap Framework.” Available: <https://getbootstrap.com/>